FundRadar

AI-Powered Funding Opportunity Intelligence Platform

Project Documentation & Roadmap

Status as of June 2026 · Prepared for the FundRadar project team

1. Executive Summary

FundRadar collects funding opportunities — grants, fellowships, scholarships, research programmes and calls for proposals — that are today scattered across hundreds of agency, foundation and university websites, and brings them into a single, continuously updated and searchable platform with an AI-powered chatbot interface.

This document is a complete snapshot of the project: what has been built, how the system works end to end, how much data we have collected so far, how the scraper will be extended to cover every source, how we will add new industries and university funding pages, where the system is hosted now and where it will move, and the remaining roadmap with the challenges and the longer-term vision.

2. The Problem and Our Solution

Researchers, students, startups and institutions struggle to find relevant funding. Opportunities are published in hundreds of different places, the information changes constantly (deadlines move, new programmes appear, eligibility is revised), and tracking it manually is slow and error-prone. Valuable opportunities are routinely missed.

FundRadar solves this by automatically collecting funding information from many sources, monitoring those sources for changes, using AI to convert messy web pages into clean structured records, and serving the result through search and a chatbot — so a user asks one question in one place instead of visiting a hundred websites.

3. System Architecture and Data Pipeline

The platform is a four-stage pipeline that turns raw websites into a clean, searchable database, wrapped by an automation loop that keeps everything fresh.

1. Agency database — the list of funding sources (agencies, foundations, universities) with their websites. Seeded from the project spreadsheet.
2. Scraper — visits each source, finds the funding-related pages, downloads them, and stores the text with a change-detection fingerprint.

3. AI extraction — reads each page and produces structured fields (amount, eligibility, deadline, research area), plus a summary and tags; normalises amounts and dates.
4. API & dashboard — serves the data over the web with filtering, and presents it in a clean dashboard (and, in future, a chatbot).

An automation job ties stages 2 and 3 together and runs on a schedule (every two days), so the database continuously reflects the latest opportunities.

Technology stack

Layer	Technology
Backend / API	Python, FastAPI, Uvicorn
Database	SQLite now (development); PostgreSQL-ready for production
Scraping	httpx + BeautifulSoup (static); Playwright planned for JavaScript sites
AI extraction	Pluggable LLM layer — Google Gemini (free tier), local Ollama, or mock
Interface	HTML/CSS/JS dashboard served by FastAPI; chatbot planned
Automation	Single pipeline command scheduled via cron or systemd timer

4. What We Have Built

Four phases are complete. The system runs end to end today: it loads agencies, scrapes sites, extracts structured data with AI, serves a professional dashboard, and can run the whole pipeline automatically on a schedule.

Phase	Component	Status
0	Foundation: config, database schema, Excel loader, pluggable LLM layer	Complete
1	Read API: endpoints for agencies & opportunities with filtering and pagination	Complete
2	Scraper: polite fetcher, funding-link discovery, text extraction, change detection	Complete
3	AI extraction: structured JSON,	Complete

	amount/date normalisation, auto-close past deadlines	
4	Monitoring: full-pipeline job with logging + every-2-days scheduling	Complete
UI	Professional dashboard: searchable, sortable, filterable tables at the root URL	Complete
2b	Playwright (JavaScript sites) + PDF extraction	Planned
5	Semantic search + AI chatbot	Planned
6	Full user-facing frontend	Planned
7	Hardening: PostgreSQL, auth, alerts, tests, scale	Planned

5. Current Status and Coverage

The database currently holds 41 funding sources spanning 18 countries and blocs. Of these, 11 have already been processed into real, current funding opportunities — 25 live opportunities in total, each with a programme name, amount where stated, deadline, eligibility, research area, summary and tags. The remaining 30 sources are loaded and queued for scraping.

Metric	Value
Funding sources in database	41
Sources with extracted opportunities	11
Opportunities extracted (real, current)	25
Countries / blocs represented	18
Open vs closed (as of June 2026)	12 open · 11 closed · 2 undated

The 11 processed sources include DST, ANRF, DBT, ICMR and BIRAC (India), and CEFIPRA, USIEF/Fulbright-Nehru, SNSF, JSPS, the European Commission (Horizon Europe) and the Alexander von Humboldt Foundation (international).

6. The Scraper: How It Works and Reaching Full Coverage

How it works today

1. Fetch — downloads an agency homepage politely: it identifies itself, respects robots.txt, rate-limits each site, and retries on failure.
2. Discover — scores every link by funding-related keywords (grant, fellowship, scholarship, call for proposals, scheme) and keeps the strong ones, ignoring login, contact and off-site links.
3. Extract — opens each kept page, strips menus and scripts to clean text, and stores it with a content fingerprint used to detect future changes.
4. Structure — the AI layer turns that text into clean fields and the normaliser tidies amounts and dates.

This was validated on the live DST website, where the scorer correctly kept the seven real funding links (Call for Proposals, India-Austria, DST-JSPS, BRICS, Fellowships) and dropped the rest.

Extracting data from the remaining sources

The same generic pipeline runs against every source — no per-site code is required for the common case. Running the full job (one command) sweeps all 41 sources. In practice some return rich results immediately, while others return little until two capabilities are added:

- **JavaScript-heavy sites:** many modern agency sites load content with JavaScript, so the plain download sees an almost-empty page. The fix is to render the page in a real headless browser (Playwright) before extracting — planned as Phase 2b.
- **PDF-only calls:** some agencies publish calls as linked PDF notifications. The scraper finds the link but cannot yet read the file. Adding PDF text extraction (Phase 2b) closes this gap.
- **Unusual labels / deep pages:** a few sites use vague labels or bury calls several clicks deep; handled by extending the keyword list and allowing a deeper crawl for specific sites.

Each extracted record will also carry a confidence indicator, so low-confidence rows can be flagged for a quick human check rather than published blindly.

7. Expanding Coverage

Adding more industries

The database is industry-agnostic by design. Adding a new industry simply means adding its funders to the source list (with a website and a category); the same scraping and AI pipeline then handles them, and the AI auto-categorises each opportunity by research domain. No code changes are needed for a standard new source.

Industry / Domain	Example funders to add
Agriculture & food	ICAR, FAO grants, agri-tech foundations

Climate & clean energy	MNRE, Clean Hydrogen Partnership, climate funds
Health & biotech	additional ICMR/DBT schemes, Wellcome, Gates Foundation
Arts, humanities & social science	ICSSR, Ford Foundation, national arts councils
Startups & innovation	Startup India, SISFS, corporate accelerators
Defence & space	DRDO, ISRO calls, national space agencies
Corporate / CSR foundations	Tata Trusts, Infosys Foundation, company CSR funds

Process to add an industry: collect the funder names and websites, add them to the agency database with an appropriate category, run the seed and the pipeline — the new opportunities then appear in the dashboard automatically.

Scraping university funding and college tie-up pages

Universities are a rich, under-used source: most maintain pages for sponsored research, fellowships, scholarships, international collaborations and MoUs / institutional tie-ups. We already hold three university sources in the database (Stanford, Bar-Ilan, IIT Hyderabad) as a starting point, and the approach generalises:

- **Treat universities as a source type:** add them with a UNIVERSITY category so they can be reported and filtered separately from agencies.
- **Point at the right pages:** university opportunities live on research-office / sponsored-research pages (often "ORSP", "Research & Development", "International Relations"), sometimes on a sub-site — the scraper targets those entry points.
- **Extend the keyword set:** add terms like sponsored research, fellowship, scholarship, exchange, collaboration, MoU, partnership and call so discovery catches both funding and tie-up pages.
- **Capture tie-ups too:** the same pages list institutional partnerships and exchange programmes, which we can store as a related record type for students seeking collaborations.
- **Curated seed list:** begin with a vetted list of universities that publicly list opportunities, then expand.

8. Hosting: Local Now, Server Later

The system is intentionally built to run the same way on a laptop and on a server, so we can develop locally now and move to a server with no rewrite.

Now — on your computer

- Runs locally with one command for the data and one for the web server; the dashboard opens at http://127.0.0.1:8000/.
- Uses a single SQLite database file — zero setup.
- Ideal for development, testing and demonstrations.
- Limitation: the scheduled scraper only runs while the computer is on.

Future — an always-on server

- A small always-on Linux server (a low-cost cloud VM) so the every-2-days job runs reliably without depending on a laptop being awake.
- Switch the database from SQLite to PostgreSQL (the code already supports this) for concurrency and scale.
- Run the scraper job on a cron or systemd timer, and the web server as a managed service, reachable from any browser via the server address.
- The full step-by-step server setup is documented in DEPLOYMENT.md.

9. What Is Currently Lacking

- JavaScript-site rendering (Playwright) and PDF reading are not yet built, so some sources return little (Phase 2b).
- Only 11 of 41 sources have been processed into opportunities so far; the rest are queued.
- No chatbot or semantic search yet — querying is via filters, not natural language (Phase 5).
- The interface is a single dashboard; a full product (accounts, saved searches, alerts) is still to come (Phase 6).
- Still on SQLite; no production database, authentication, or automated tests yet (Phase 7).
- No formal change-log/audit table recording exactly what changed between runs.

10. Challenges

- **Heterogeneous websites:** every agency structures its site differently, so discovery must stay general yet accurate.
- **JavaScript & PDFs:** a large share of real content is rendered by JavaScript or locked in PDFs, requiring heavier tooling.
- **Data quality & normalisation:** amounts and deadlines appear in many formats and currencies (lakh/crore, $, €, CHF) and must be standardised reliably.
- **Freshness at scale:** keeping hundreds of sources current means efficient change detection and scheduling.
- **Politeness & access:** respecting robots.txt and rate limits, and handling sites that block automated access.
- **AI cost and limits:** staying within free/low-cost LLM tiers while maintaining extraction quality.

- **Deduplication & multilingual content:** the same call can appear on several sites and in several languages.

11. The Horizon

The foundation, scraping and AI extraction are working. The path ahead turns this engine into a complete, self-running product.

Next step	What it delivers	How we achieve it
Phase 2b	Full coverage of JS sites and PDFs	Add Playwright rendering and a PDF text extractor to the scraper
Phase 5	Natural-language chatbot search	Add embeddings + a vector database and answer only from verified data (RAG)
Phase 6	User-facing product	Build a React/Next.js frontend with search, alerts and saved opportunities
Phase 7	Production hardening	Move to PostgreSQL, add accounts, deadline alerts, tests, scale on the server
Growth	More sources & industries	Expand the source list across industries and universities using the existing pipeline

Vision: a secure, scalable platform that any researcher, student, startup or institution can ask, in plain language, "what funding is open for me?" — and get an accurate, current answer drawn from a continuously monitored database of opportunities worldwide.

12. Quick Reference — Running the System

From the project folder, with the environment activated:

Command	What it does
python -m app.ingest.seed_agencies	Load the 41 sources into the database
python -m app.ingest.load_demo_opportunities	Load the 25 real demo opportunities
python -m uvicorn app.api.main:app	Start the web server (dashboard at http://127.0.0.1:8000/)
python -m app.scraper.run --limit 5	Live-scrape the top 5 sources
python -m app.run_pipeline	Full automated pass: scrape all sources + AI

	extract
python demo_scraper.py	Watch the scraper logic step by step (offline)

Reference documents in the project: README.md (commands), ROADMAP.md (full plan & status), PROJECT_GUIDE.md (plain-English guide), DEMO_RESULTS.md (presentation report), DEPLOYMENT.md (server & scheduling).